

Deep Learning

12b. Recurrent Neural Networks (contd.)

Dr. Konda Reddy Mopuri
Dept. of AI, IIT Hyderabad
Jan-May 2026

So far...

① Perceptron, MLP, Gradient Descent (Backpropagation)

So far...

- ① Perceptron, MLP, Gradient Descent (Backpropagation)
- ② CNNs

So far...

- ① Perceptron, MLP, Gradient Descent (Backpropagation)
- ② CNNs
- ③ 'Feedforward Neural networks'

So far...

- ① Perceptron, MLP, Gradient Descent (Backpropagation)
- ② CNNs
- ③ 'Feedforward Neural networks'
- ④ Sequence learning problems, RNNs, BPTT

- ① Consider a finite response model with bounded inputs

$$Y(t) = f(X(t - i); i = 0, \dots, K)$$

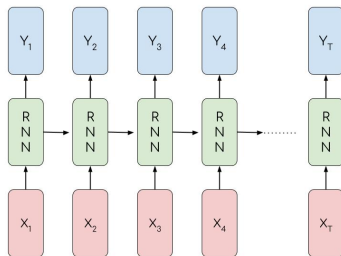
Stability Analysis

- ① Consider a finite response model with bounded inputs
$$Y(t) = f(X(t - i); i = 0, \dots, K)$$
- ② Will the output blow up?

Stability Analysis

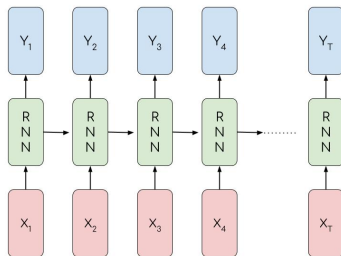
- ① Consider a finite response model with bounded inputs
$$Y(t) = f(X(t - i); i = 0, \dots, K)$$
- ② Will the output blow up?
- ③ The o/p will be bounded if $f(\cdot)$ is BIBO stable

Will the RNN also be BIBO stable?



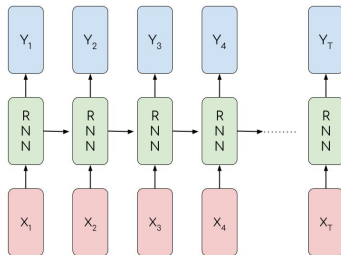
Will the RNN also be BIBO stable?

- Let's analyze the hidden layer h_t (since it carries the relevant information)



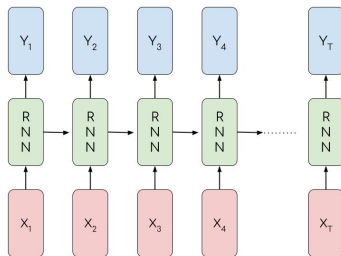
Will the RNN also be BIBO stable?

- 1 Let's analyze the hidden layer h_t (since it carries the relevant information)
- 2 Assume a single hidden layer and linear (identity) activation



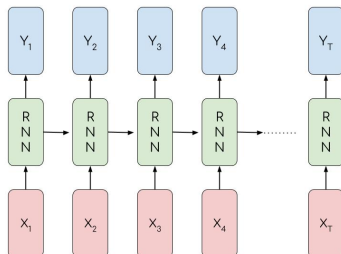
Will the RNN also be BIBO stable?

① $z_t = W_h h_{t-1} + W_x x_t$ and $h_t = z_t$



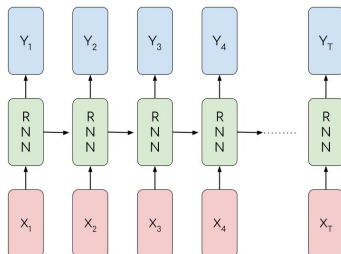
Will the RNN also be BIBO stable?

- ① $z_t = W_h h_{t-1} + W_x x_t$ and $h_t = z_t$
- ② $h_k = W_h h_{k-1} + W_x x_k$ (k denotes the time)



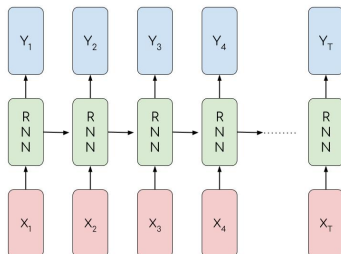
Will the RNN also be BIBO stable?

- ① $z_t = W_h h_{t-1} + W_x x_t$ and $h_t = z_t$
- ② $h_k = W_h h_{k-1} + W_x x_k$ (k denotes the time)
- ③ We know $h_{k-1} = W_h h_{k-2} + W_x x_{k-1}$



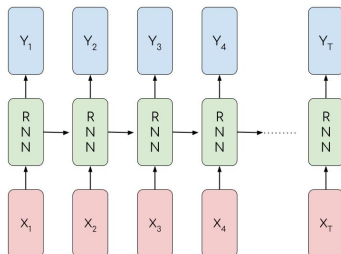
Will the RNN also be BIBO stable?

- ① $z_t = W_h h_{t-1} + W_x x_t$ and $h_t = z_t$
- ② $h_k = W_h h_{k-1} + W_x x_k$ (k denotes the time)
- ③ We know $h_{k-1} = W_h h_{k-2} + W_x x_{k-1}$
- ④ $h_k = W_h^2 h_{k-2} + W_h W_x x_{k-1} + W_x x_k$



Will the RNN also be BIBO stable?

- ① $z_t = W_h h_{t-1} + W_x x_t$ and $h_t = z_t$
- ② $h_k = W_h h_{k-1} + W_x x_k$ (k denotes the time)
- ③ We know $h_{k-1} = W_h h_{k-2} + W_x x_{k-1}$
- ④ $h_k = W_h^2 h_{k-2} + W_h W_x x_{k-1} + W_x x_k$
- ⑤ $h_k = W_h^{k+1} h_{-1} + W_h^k W_x x_0 + W_h^{k-1} W_x x_1 + W_h^{k-2} W_x x_2 + \dots$



Linear system analysis

$$\textcircled{1} \quad h_k = W_h^{k+1}h_{-1} + W_h^k W_x x_0 + W_h^{k-1}W_x x_1 + W_h^{k-2}W_x x_2 + \dots$$

Linear system analysis

- ① $h_k = W_h^{k+1}h_{-1} + W_h^k W_x x_0 + W_h^{k-1}W_x x_1 + W_h^{k-2}W_x x_2 + \dots$
- ② Response at time k to an input at time 0 (with no other i/p's and zero initial condition)

Linear system analysis

- ① $h_k = W_h^{k+1}h_{-1} + W_h^k W_x x_0 + W_h^{k-1}W_x x_1 + W_h^{k-2}W_x x_2 + \dots$
- ② Response at time k to an input at time 0 (with no other i/ps and zero initial condition)
- ③ If this blows up, the system blows up

Linear system analysis

- ① $h_k = W_h^{k+1}h_{-1} + W_h^k W_x x_0 + W_h^{k-1}W_x x_1 + W_h^{k-2}W_x x_2 + \dots$
- ② Response at time k to an input at time 0 (with no other i/ps and zero initial condition)
- ③ If this blows up, the system blows up
- ④ if this does not blow up, the responses to later i/ps will also not blow up!

Linear system analysis

- ① $h_k = W_h^{k+1}h_{-1} + W_h^k W_x x_0 + W_h^{k-1} W_x x_1 + W_h^{k-2} W_x x_2 + \dots$
- ② What is ideal behavior?

Linear system analysis

- ① $h_k = W_h^{k+1}h_{-1} + W_h^k W_x x_0 + W_h^{k-1} W_x x_1 + W_h^{k-2} W_x x_2 + \dots$
- ② What is ideal behavior?
- ③ Something of i/p at time 0 must persist forever (the h_k must neither blow up nor fade out)

Linear system analysis

- ① Consider a simple, scalar, linear recursion
- ② $h(t) = W \cdot h(t - 1) + c \cdot x(t)$

Linear system analysis

- ① Consider a simple, scalar, linear recursion
- ② $h(t) = W \cdot h(t-1) + c \cdot x(t)$
- ③ $h_0(t) = w^t \cdot c \cdot x(0)$ (response to a single i/p at time 0)

Linear system analysis

- ① Consider a simple, scalar, linear recursion
- ② $h(t) = W \cdot h(t-1) + c \cdot x(t)$
- ③ $h_0(t) = w^t \cdot c \cdot x(0)$ (response to a single i/p at time 0)
- ④ When will this blow up?

Linear system analysis

- ① Consider a simple, scalar, linear recursion
- ② $h(t) = W \cdot h(t-1) + c \cdot x(t)$
- ③ $h_0(t) = w^t \cdot c \cdot x(0)$ (response to a single i/p at time 0)
- ④ When will this blow up?
- ⑤ When $|w| > 1$

Linear system analysis

- ① Consider a simple, scalar, linear recursion
- ② $h(t) = W \cdot h(t-1) + c \cdot x(t)$
- ③ $h_0(t) = w^t \cdot c \cdot x(0)$ (response to a single i/p at time 0)
- ④ When will this blow up?
- ⑤ When $|w| > 1$
- ⑥ When $|w| < 1$, the response falls to 0

Linear system analysis

① Consider vector linear recursion

② $h(t) = Wh(t - 1) + Cx(t)$

Linear system analysis

- ① Consider vector linear recursion
- ② $h(t) = Wh(t-1) + Cx(t)$
- ③ $h_0(t) = W^t Cx(0)$ (response vector to a single i/p at time 0)

Linear system analysis

- ① Consider vector linear recursion
- ② $h(t) = Wh(t-1) + Cx(t)$
- ③ $h_0(t) = W^t Cx(0)$ (response vector to a single i/p at time 0)
- ④ We can write $W = U\Lambda U^{-1} = U \begin{bmatrix} \lambda_1 & 0 & 0 \\ 0 & \lambda_2 & 0 \\ 0 & 0 & \ddots \end{bmatrix} U^{-1}$

Linear system analysis

- ① Consider vector linear recursion
- ② $h(t) = Wh(t-1) + Cx(t)$
- ③ $h_0(t) = W^t Cx(0)$ (response vector to a single i/p at time 0)

④ We can write $W = U\Lambda U^{-1} = U \begin{bmatrix} \lambda_1 & 0 & 0 \\ 0 & \lambda_2 & 0 \\ 0 & 0 & \ddots \end{bmatrix} U^{-1}$

⑤ $W^t = U \begin{bmatrix} \lambda_1^t & 0 & 0 \\ 0 & \lambda_2^t & 0 \\ 0 & 0 & \ddots \end{bmatrix} U^{-1} = \lambda_1^t U \begin{bmatrix} 1 & 0 & 0 \\ 0 & \frac{\lambda_2^t}{\lambda_1^t} & 0 \\ 0 & 0 & \ddots \end{bmatrix} U^{-1}$

Linear system analysis

① $h_0(t) = W^t C x(0)$ (response vector to a single i/p at time 0)

$$\textcircled{2} \quad W^t = U \begin{bmatrix} \lambda_1^t & 0 & 0 \\ 0 & \lambda_2^t & 0 \\ 0 & 0 & \ddots \end{bmatrix} U^{-1} = \lambda_1^t U \begin{bmatrix} 1 & 0 & 0 \\ 0 & \frac{\lambda_2^t}{\lambda_1^t} & 0 \\ 0 & 0 & \ddots \end{bmatrix} U^{-1}$$

Linear system analysis

① $h_0(t) = W^t C x(0)$ (response vector to a single i/p at time 0)

②
$$W^t = U \begin{bmatrix} \lambda_1^t & 0 & 0 \\ 0 & \lambda_2^t & 0 \\ 0 & 0 & \ddots \end{bmatrix} U^{-1} = \lambda_1^t U \begin{bmatrix} 1 & 0 & 0 \\ 0 & \frac{\lambda_2^t}{\lambda_1^t} & 0 \\ 0 & 0 & \ddots \end{bmatrix} U^{-1}$$

③ For any i/p, magnitude of the hidden vector will expand or contract according to the t^{th} power of the largest eigen value of the recurrent matrix

Linear system analysis

① $h_0(t) = W^t C x(0)$ (response vector to a single i/p at time 0)

②
$$W^t = U \begin{bmatrix} \lambda_1^t & 0 & 0 \\ 0 & \lambda_2^t & 0 \\ 0 & 0 & \ddots \end{bmatrix} U^{-1} = \lambda_1^t U \begin{bmatrix} 1 & 0 & 0 \\ 0 & \frac{\lambda_2^t}{\lambda_1^t} & 0 \\ 0 & 0 & \ddots \end{bmatrix} U^{-1}$$

③ For any i/p, magnitude of the hidden vector will expand or contract according to the t^{th} power of the largest eigen value of the recurrent matrix

Linear system analysis

① $h_0(t) = W^t C x(0)$ (response vector to a single i/p at time 0)

②
$$W^t = U \begin{bmatrix} \lambda_1^t & 0 & 0 \\ 0 & \lambda_2^t & 0 \\ 0 & 0 & \ddots \end{bmatrix} U^{-1} = \lambda_1^t U \begin{bmatrix} 1 & 0 & 0 \\ 0 & \frac{\lambda_2^t}{\lambda_1^t} & 0 \\ 0 & 0 & \ddots \end{bmatrix} U^{-1}$$

③ For any i/p, magnitude of the hidden vector will expand or contract according to the t^{th} power of the largest eigen value of the recurrent matrix

④ If $|\lambda_1| > 1$ it will blow up, otherwise it will shrink to 0

Linear system analysis - Summary

- ① In linear systems, long-term behavior depends entirely on the eigenvalues of the recurrent weight matrix

Linear system analysis - Summary

- ① In linear systems, long-term behavior depends entirely on the eigenvalues of the recurrent weight matrix
- ② If the $|\lambda_1| > 1$, system will blow up

Linear system analysis - Summary

- ① In linear systems, long-term behavior depends entirely on the eigenvalues of the recurrent weight matrix
- ② If the $|\lambda_1| > 1$, system will blow up
- ③ If the $|\lambda_1| < 1$, response will vanish

Linear system analysis - Summary

- ① In linear systems, long-term behavior depends entirely on the eigenvalues of the recurrent weight matrix
- ② If the $|\lambda_1| > 1$, system will blow up
- ③ If the $|\lambda_1| < 1$, response will vanish
- ④ Same behavior for complex eigenvalues also, but with oscillatory response

Linear system analysis - Summary

- ① In linear systems, long-term behavior depends entirely on the eigenvalues of the recurrent weight matrix
- ② If the $|\lambda_1| > 1$, system will blow up
- ③ If the $|\lambda_1| < 1$, response will vanish
- ④ Same behavior for complex eigenvalues also, but with oscillatory response
- ⑤ Rate of blowing up or vanishing depends only on the eigenvalues, but not on i/p

With activation function - Sigmoid, Tanh, and ReLU

- ① Final value depends only on b and W , but not on x

With activation function - Sigmoid, Tanh, and ReLU

- ① Final value depends only on b and W , but not on x
- ② However, system with Tanh remembers x for longer duration than with the Sigmoid

With activation function - Sigmoid, Tanh, and ReLU

- ① Final value depends only on b and W , but not on x
- ② However, system with Tanh remembers x for longer duration than with the Sigmoid
- ③ What about ReLU?

With activation function - Sigmoid, Tanh, and ReLU

- ① Final value depends only on b and W , but not on x
- ② However, system with Tanh remembers x for longer duration than with the Sigmoid
- ③ What about ReLU?
- ④ if $w > 1$, for $x > 0$, response blows up

With activation function - Sigmoid, Tanh, and ReLU

- ① Final value depends only on b and W , but not on x
- ② However, system with Tanh remembers x for longer duration than with the Sigmoid
- ③ What about ReLU?
- ④ if $w > 1$, for $x > 0$, response blows up
- ⑤ Response dies for $x < 0$

Stability Analysis - Summary

- 1 Bipolar activation functions have better memory behavior

Stability Analysis - Summary

- ① Bipolar activation functions have better memory behavior
- ② However, sensitive to the Eigenvalues of W and b

Stability Analysis - Summary

- ① Bipolar activation functions have better memory behavior
- ② However, sensitive to the Eigenvalues of W and b
- ③ Best case memory is short

Stability Analysis - Summary

- ① Bipolar activation functions have better memory behavior
- ② However, sensitive to the Eigenvalues of W and b
- ③ Best case memory is short
- ④ Forgets in exponential manner

- ① RNNs, in principle, retain information from the infinite past

RNNs - So far

- ① RNNs, in principle, retain information from the infinite past
- ② However, in practice, they either blow up or lose the information

RNNs - So far

- ① RNNs, in principle, retain information from the infinite past
- ② However, in practice, they either blow up or lose the information
- ③ Memory (for how long and what is retained) depends on the activation of the hidden units - is not all that great!

RNNs - So far

$$\textcircled{1} \quad \nabla_{f_k} L = \nabla L \cdot \nabla f_N \cdot W_N \cdot \nabla f_{N-1} \cdot W_{N-1} \dots \nabla f_{K+1} \cdot W_{K+1}$$

- ① $\nabla_{f_k} L = \nabla L \cdot \nabla f_N \cdot W_N \cdot \nabla f_{N-1} \cdot W_{N-1} \dots \nabla f_{K+1} \cdot W_{k+1}$
- ② The product will expand/shrink in the directions in which the singular values of the weight matrices are more/less than 1

- ① $\nabla_{f_k} L = \nabla L \cdot \nabla f_N \cdot W_N \cdot \nabla f_{N-1} \cdot W_{N-1} \dots \nabla f_{K+1} \cdot W_{k+1}$
- ② The product will expand/shrink in the directions in which the singular values of the weight matrices are more/less than 1
- ③ Results in insignificant/unstable updates

Handling long-term dependencies

- ① Replaces the weights and activation functions with something that doesn't fade/blow up

Handling long-term dependencies

- ① Replaces the weights and activation functions with something that doesn't fade/blow up
- ② Networks that 'retain' useful memory for an arbitrarily long period and 'recall' on demand

Handling long-term dependencies

- ① Replaces the weights and activation functions with something that doesn't fade/blow up
- ② Networks that 'retain' useful memory for an arbitrarily long period and 'recall' on demand
- ③ I/p based determination of whether it must be remembered

Handling long-term dependencies

- ① Replaces the weights and activation functions with something that doesn't fade/blow up
- ② Networks that 'retain' useful memory for an arbitrarily long period and 'recall' on demand
- ③ I/p based determination of whether it must be remembered
- ④ A switch based on i/p controls

Handling long-term dependencies

- ① Replaces the weights and activation functions with something that doesn't fade/blow up
- ② Networks that 'retain' useful memory for an arbitrarily long period and 'recall' on demand
- ③ I/p based determination of whether it must be remembered
- ④ A switch based on i/p controls
- ⑤ memory at $k \approx C(x_0) \cdot \sigma(x_1) \cdot \sigma(x_2) \dots \sigma(x_k)$

Handling long-term dependencies

① Architectural modifications to RNNs

Handling long-term dependencies

① Architectural modifications to RNNs

- LSTM (1997 by Sepp Hochreiter and Jürgen Schmidhuber; Improved by Gers et al. in 2000)

① Architectural modifications to RNNs

- LSTM (1997 by Sepp Hochreiter and Jürgen Schmidhuber; Improved by Gers et al. in 2000)
- GRU (Cho et al. 2014)

① Long Short-Term Memory

- ① Long Short-Term Memory
- ② At a time 't', **hidden state** $h^{(t)}$ and **cell state** $c^{(t)}$

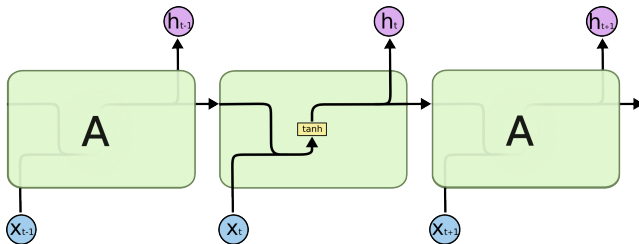
- ① Long Short-Term Memory
- ② At a time 't', **hidden state** $h^{(t)}$ and **cell state** $c^{(t)}$
 - Cell stores long-term information

- ① Long Short-Term Memory
- ② At a time 't', **hidden state** $h^{(t)}$ and **cell state** $c^{(t)}$
 - Cell stores long-term information
 - LSTM can **erase, write, and read** information from the cell

- ① Long Short-Term Memory
- ② At a time 't', **hidden state** $h^{(t)}$ and **cell state** $c^{(t)}$
 - Cell stores long-term information
 - LSTM can **erase, write, and read** information from the cell
- ③ What to erase/write/read is controlled by corresponding **gates**

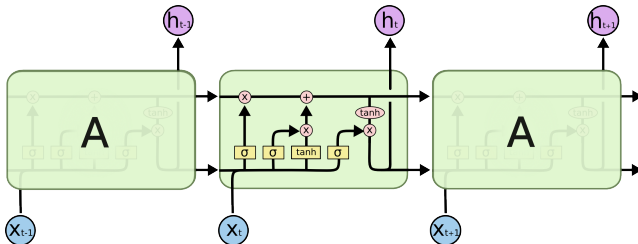
- ① Long Short-Term Memory
- ② At a time 't', **hidden state** $h^{(t)}$ and **cell state** $c^{(t)}$
 - Cell stores long-term information
 - LSTM can **erase, write, and read** information from the cell
- ③ What to erase/write/read is controlled by corresponding **gates**
 - At time t, elements of the gates can be 0 (closed), 1 (open), or in-between

- ① Long Short-Term Memory
- ② At a time 't', **hidden state** $h^{(t)}$ and **cell state** $c^{(t)}$
 - Cell stores long-term information
 - LSTM can **erase, write, and read** information from the cell
- ③ What to erase/write/read is controlled by corresponding **gates**
 - At time t, elements of the gates can be 0 (closed), 1 (open), or in-between
 - Gates are dynamically computed based on the context



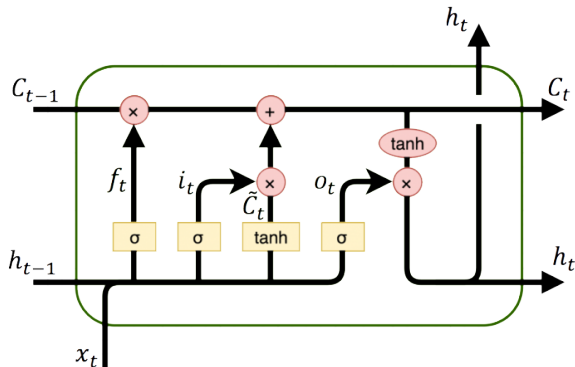
RNNs are chain of repeating moduels. Basic RNN (Colah's blog)

LSTM



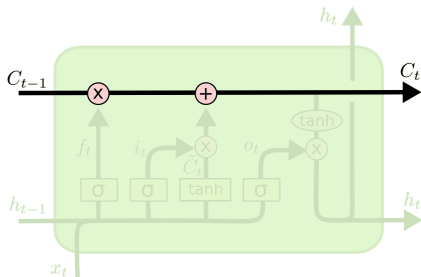
RNNs are chain of repeating moduels. LSTM (Colah's blog)

LSTM



The LSTM node. (Colah's blog)

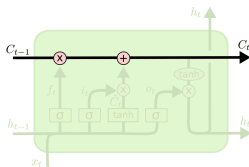
LSTM: The cell state



Cell state in LSTM (Colah's blog)

LSTM: The cell state

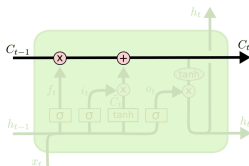
- 1 Info. can flow through unchanged



Cell state in LSTM (Colah's blog)

LSTM: The cell state

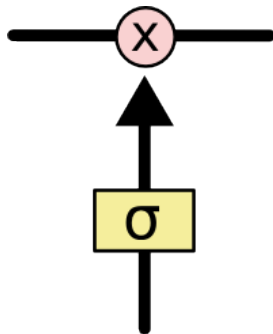
- 1 Info. can flow through unchanged
- 2 Gates can add/remove information to cell state



Cell state in LSTM (Colah's blog)

LSTM: The gates

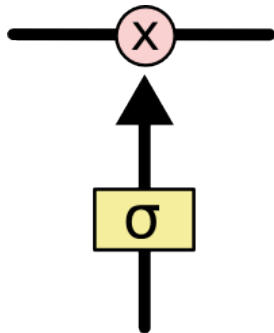
- ① Sigmoid neural nets (o/p numbers in $[0, 1]$)



Cell state in LSTM (Colah's blog)

LSTM: The gates

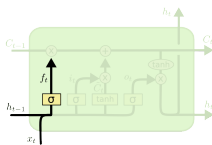
- ① Sigmoid neural nets (o/p numbers in $[0, 1]$)
- ② Point-wise multiplication operation



Cell state in LSTM (Colah's blog)

LSTM: The forget gate

- ① Decides what to throw away from cell state (e.g. forgetting the gender of old subject in light of a new one)

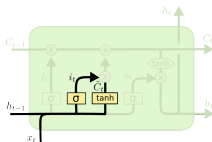


$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

Forget gate in LSTM (Colah's blog)

LSTM: The input gate

- 1 Next is to decide what new to store in cell state (e.g. add the gender of a new subject)



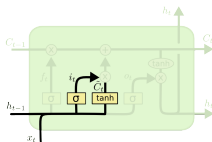
$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

Input gate in LSTM (Colah's blog)

LSTM: The input gate

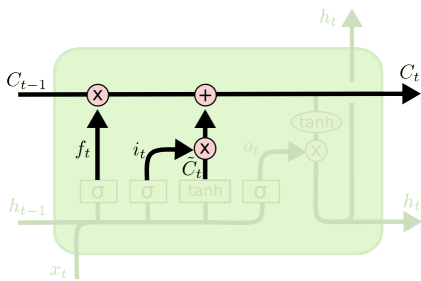
- ① Next is to decide what new to store in cell state (e.g. add the gender of a new subject)
- ② Done in two steps
 - input gate decides what to update
 - A tanh layer creates a candidate cell state



$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$
$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

Input gate in LSTM (Colah's blog)

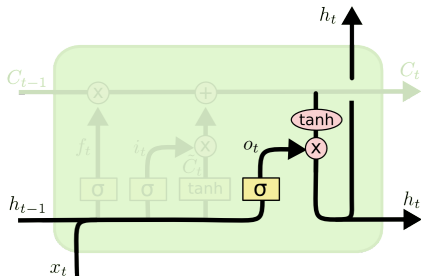
LSTM: The cell state update



$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

Cell state update in LSTM (Colah's blog)

LSTM: The output



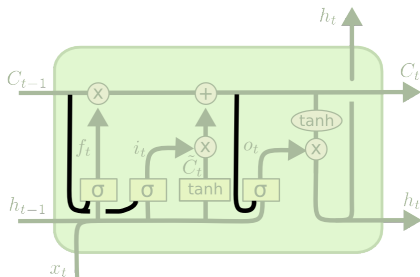
$$o_t = \sigma(W_o [h_{t-1}, x_t] + b_o)$$

$$h_t = o_t * \tanh(C_t)$$

Output computation in LSTM (Colah's blog)

e.g. may be a verb that is coming next in case of a language model

LSTM variant: Peephole connections



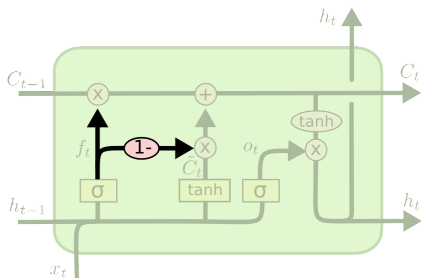
$$f_t = \sigma(W_f \cdot [C_{t-1}, h_{t-1}, x_t] + b_f)$$

$$i_t = \sigma(W_i \cdot [C_{t-1}, h_{t-1}, x_t] + b_i)$$

$$o_t = \sigma(W_o \cdot [C_t, h_{t-1}, x_t] + b_o)$$

Variant with gates looking into the Cell state in LSTM by Ger et al. (Colah's blog)

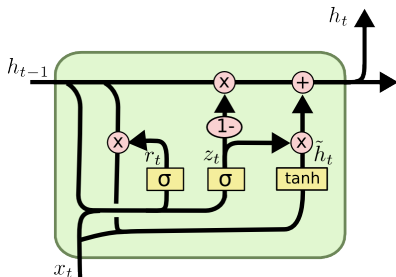
LSTM variant: Coupled i/p and forget gates



$$C_t = f_t * C_{t-1} + (1 - f_t) * \tilde{C}_t$$

Variant with coupled input and forget gates. (Colah's blog)

LSTM → GRU



$$z_t = \sigma(W_z \cdot [h_{t-1}, x_t])$$

$$r_t = \sigma(W_r \cdot [h_{t-1}, x_t])$$

$$\tilde{h}_t = \tanh(W \cdot [r_t * h_{t-1}, x_t])$$

$$h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t$$

Gated Recurrent Unit (Colah's blog)

LSTM: handling the vanishing gradients

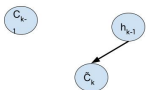
① Via the gates!

LSTM: handling the vanishing gradients



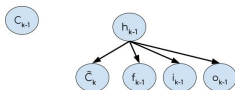
- 1 Computational graph at time $k - 1$

LSTM: handling the vanishing gradients



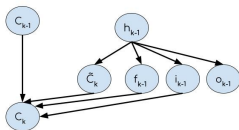
$$\textcircled{1} \quad \tilde{c}_k = \tanh(W_c[h_{t-1}, x_t] + b_c)$$

LSTM: handling the vanishing gradients



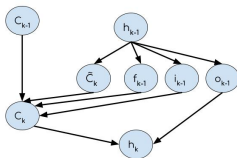
① All the gates

LSTM: handling the vanishing gradients



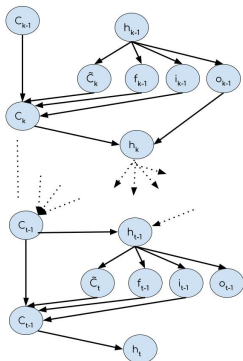
① Next cell state

LSTM: handling the vanishing gradients



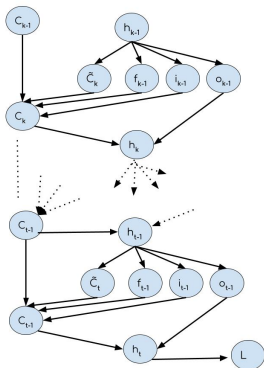
① Next hidden state

LSTM: handling the vanishing gradients



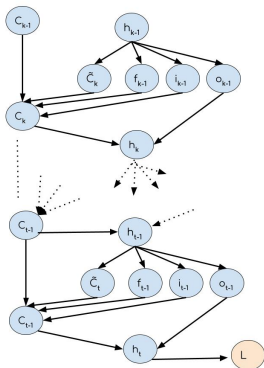
① Running till time step 't'

LSTM: handling the vanishing gradients



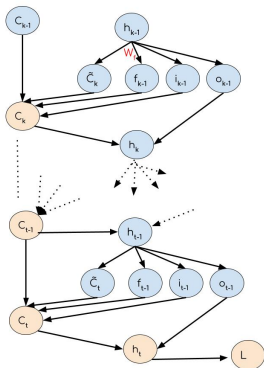
- 1 Consider loss computation

LSTM: handling the vanishing gradients



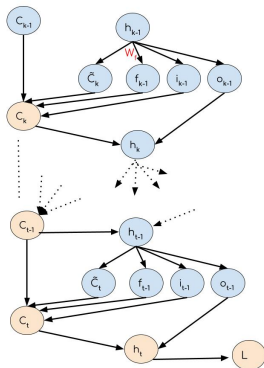
- 1 Let's know if the gradient flows to an arbitrary time step 'k'

LSTM: handling the vanishing gradients



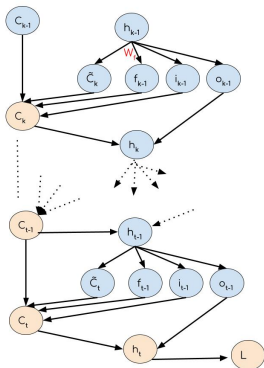
- 1 Specifically, let's consider if gradient flows to W_f through C_k

LSTM: handling the vanishing gradients



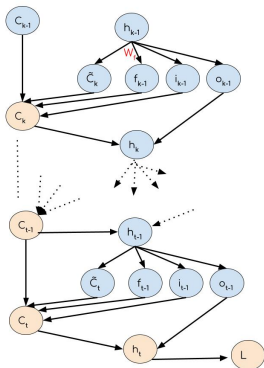
- 1 Specifically, let's consider if gradient flows to W_f through C_k
- 2 Note that there are multiple paths between L and C_k (but, consider one such path as highlighted)

LSTM: handling the vanishing gradients



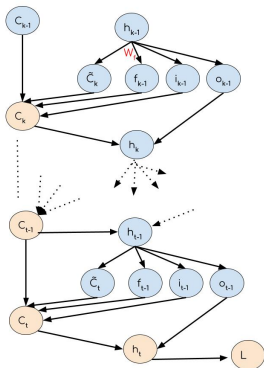
① Grad =
$$\frac{\partial L}{\partial h_t} \frac{\partial h_t}{\partial C_t} \frac{\partial C_t}{\partial C_{t-1}} \cdots \frac{\partial C_{k+1}}{\partial C_k}$$

LSTM: handling the vanishing gradients



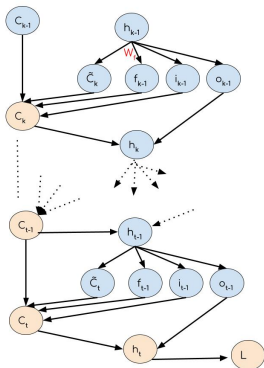
- 1 Grad = $\frac{\partial L}{\partial h_t} \frac{\partial h_t}{\partial C_t} \frac{\partial C_t}{\partial C_{t-1}} \cdots \frac{\partial C_{k+1}}{\partial C_k}$
- 2 $\frac{\partial L}{\partial h_t}$ doesn't vanish (no intermediate nodes)

LSTM: handling the vanishing gradients



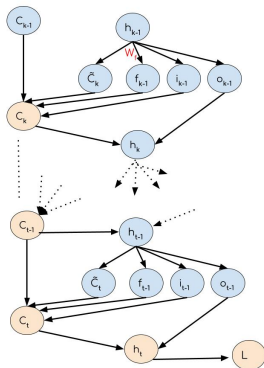
- ① $\text{Grad} = \frac{\partial L}{\partial h_t} \frac{\partial h_t}{\partial C_t} \frac{\partial C_t}{\partial C_{t-1}} \cdots \frac{\partial C_{k+1}}{\partial C_k}$
- ② $\frac{\partial L}{\partial h_t}$ doesn't vanish (no intermediate nodes)
- ③ $h_t = o_t \odot \sigma(C_t)$

LSTM: handling the vanishing gradients



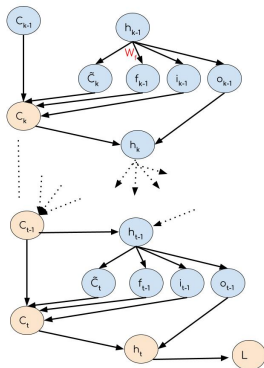
- 1 Grad = $\frac{\partial L}{\partial h_t} \frac{\partial h_t}{\partial C_t} \frac{\partial C_t}{\partial C_{t-1}} \cdots \frac{\partial C_{k+1}}{\partial C_k}$
- 2 $\frac{\partial L}{\partial h_t}$ doesn't vanish (no intermediate nodes)
- 3 $h_t = o_t \odot \sigma(C_t)$
- 4 $\rightarrow \frac{\partial h_t}{\partial C_t} = \mathbb{D}(o_t \odot \sigma'(C_t))$
 (diagonal matrix)

LSTM: handling the vanishing gradients



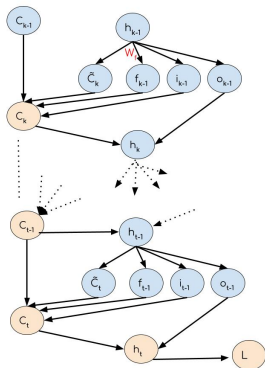
$$① \quad C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$$

LSTM: handling the vanishing gradients



- ① $C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$
- ② Note that $\tilde{C}_t$ depends on C_{t-1} , and for simplicity assume the gradient from that term vanishes

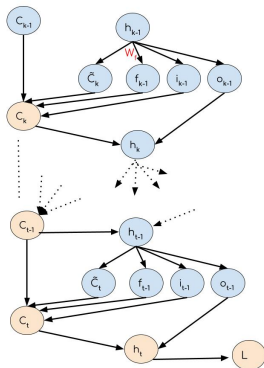
LSTM: handling the vanishing gradients



- ① $C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$
- ② Note that $\tilde{C}_t$ depends on C_{t-1} , and for simplicity assume the gradient from that term vanishes
- ③ Grad =

$$\frac{\partial L}{\partial h_t} \frac{\partial h_t}{\partial C_t} \frac{\partial C_t}{\partial C_{t-1}} \cdots \frac{\partial C_{k+1}}{\partial C_k}$$

LSTM: handling the vanishing gradients



- ① $C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$
- ② Note that $\tilde{C}_t$ depends on C_{t-1} , and for simplicity assume the gradient from that term vanishes
- ③ Grad = $\frac{\partial L}{\partial h_t} \frac{\partial h_t}{\partial C_t} \frac{\partial C_t}{\partial C_{t-1}} \cdots \frac{\partial C_{k+1}}{\partial C_k}$
- ④ Grad = $L' \cdot \mathbb{D}(o_t \odot \sigma'(C_t)) \mathbb{D}(f_t) \cdot \mathbb{D}(f_{t-1}) \cdots \mathbb{D}(f_{k+1})$

LSTM: handling the vanishing gradients

$$\textcircled{1} \text{ Grad} = \mathbf{L}' \cdot \mathbb{D}(o_t \odot \sigma'(C_t)) \mathbb{D}(f_t) \cdot \mathbb{D}(f_{t-1}) \dots \mathbb{D}(f_{k+1})$$

LSTM: handling the vanishing gradients

- ① $\text{Grad} = \mathbf{L}' \cdot \mathbb{D}(o_t \odot \sigma'(C_t)) \mathbf{D}(f_t) \cdot \mathbf{D}(f_{t-1}) \dots \mathbf{D}(f_{k+1})$
- ② Red term vanishes only if during the forward pass this product caused the information to vanish (by the time 't')!

LSTM: handling the vanishing gradients

- ① $\text{Grad} = L' \cdot \mathbb{D}(o_t \odot \sigma'(C_t)) \mathbb{D}(f_t) \cdot \mathbb{D}(f_{t-1}) \dots \mathbb{D}(f_{k+1})$
- ② Red term vanishes only if during the forward pass this product caused the information to vanish (by the time 't')!
- ③ That means, gradient will vanish only if dependency in the forward pass vanishes! (which makes sense)

LSTM: handling the vanishing gradients

- ① $\text{Grad} = L' \cdot \mathbb{D}(o_t \odot \sigma'(C_t)) \mathbb{D}(f_t) \cdot \mathbb{D}(f_{t-1}) \dots \mathbb{D}(f_{k+1})$
- ② Red term vanishes only if during the forward pass this product caused the information to vanish (by the time 't')!
- ③ That means, gradient will vanish only if dependency in the forward pass vanishes! (which makes sense)
- ④ Gates do the same regulation in backward pass as they do in the forward

① Dominated until 2015

- ① Dominated until 2015
- ② Driven applications such as handwriting recognition, ASR, Machine translation, Parsing, image captioning, VQA, etc.

- ① Dominated until 2015
- ② Driven applications such as handwriting recognition, ASR, Machine translation, Parsing, image captioning, VQA, etc.
- ③ Attention and Transformers are becoming more popular lately